St Joseph's University

36, Langford Road, Langford Gardens, Bengaluru – 560027

Department of Computer Science and Applications

Project Report submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF COMPUTER APPLICATION

2025 – 2026

NGO Management & Community Support Web Platform

Submitted by:

Aniketh B(232BCAA61)

Under the Guidance of:

Dr Mrinmoyee Bhattacharya

Ms Saranya M

St Joseph's University

36, Langford Road, Langford Gardens, Bengaluru – 560027

Department of Computer Science

CERTIFICATE OF COMPLETION

This is to certify that the project entitled "**NGO Management & Community Support Web Platform**" has been satisfactorily completed by **Aniketh B (232BCAA61)** in partial fulfilment of the award of the Bachelor of Computer Applications degree requirements prescribed by St Joseph's University, Bengaluru during the academic year 2025 – 2026.

Guide Signature

Head of the Department:

External Examiner Signature

Internal Examiner Signature

YASWANTH RURAL DEVELOPMENT SOCIETY

(Registered under Society Registration Act, XXI of 1860 (Reg no.388/1999 & Income Tax Act u/s 12-A 80G Exemption)

D.No. 25/146, Valliyappa Nagar, Greamspet Post, Chittoor District – 517 002, A.P., India

Mobile: +91 9949 961938
+91 8106 301966

E-Mail: yrds.bharathi@gmail.com

Mrs. M. Bharathi
President

Date: _____

To,
Aniketh B
Subject: Internship Offer for Web Development under Mentorship

Dear Aniketh,

We are pleased to offer you an internship with Yaswanth rural development society, a non-profit organization dedicated to help and uplift rural areas

Internship Details:
Position: Web Development Intern
Mentorship: Under NGO Mentorship Program
Duration: 1/12/2025 to 28/2/2026
Mode: [Online/Offline/Hybrid]
Stipend: Stipend may be provicded based on performance.

Roles & Responsibilities:
Design, develop, and maintain the NGO's poject Ensure user-friendly navigation, responsive design, and SEO optimization.Work under Mentorship guidance to implement best practices.Coordinate with NGO team to update events, activities, and campaigns online. Working Hours: Flexible, approx. [4 hours per week].

Benefits:
1. Internship Certificate on successful completion.
2. Hands-on experience in web development under Mentors
3. Exposure to NGO operations and contribution to social causes.

We look forward to working with you. Please confirm your acceptance by replying to this email/letter.

Warm Regards,

NC Yashwant
Chief Executive Officer
Yaswanth Rural Development Society

Yaswanth Rural Development Society
Reg No:388/1999
Chittoor
Cell:9538026060

St Joseph's University

36, Langford Road, Langford Gardens, Bengaluru – 560027

Department of Computer Science and Applications

DECLARATION

I, **Aniketh B (232BCAA61)** hereby declare that the project work **"NGO Management & Community Support Web Platform"** is an original project work carried out by myself, under the guidance of **Dr Mrinmoyee Bhattacharya and Ms Saranya M**. This project work has not been submitted earlier either to any University / Institution or any other body for the fulfilment of the requirement of a course of study.

Student Signatures with Date

Acknowledgement

We take this opportunity to express our profound and whole-hearted gratitude to Rev Fr Dr Victor Lobo SJ, Vice-Chancellor, St Joseph's University, Bengaluru, and Dr A M Bojamma, Dean, St Joseph's Institute of Information Technology (SJIIT) for providing the necessary facilities that enabled us to complete our project successfully.

We would like to express our deep sense of gratitude and sincere thanks to Dr. B G Prashanti, Associate Dean and Head of Department, for her valuable advice, towards the successful execution of this project.

We would like to express our utmost gratitude to our respected guide Dr Mrinmoyee Bhattacharya for her constant guidance, support and encouragement in aiding us to complete this project seamlessly.

We convey our heartfelt thanks to all teaching and non-teaching staff members of the Computer Science department who offered us constant motivation and counsel.

We once again take this opportunity to thank everyone for their continuous encouragement. Last but not the least, we convey our appreciation to all our friends and family who were with us to share our happiness and agony

1. ABSTRACT 1

2. INTRODUCTION 2

2.1 Problem Definition..... 3

2.2 Scope of the Project 3

2.3 Objectives 4

3. REQUIREMENT SPECIFICATION 5

3.1 Hardware Requirements 5

3.2 Software Requirements 6

4. DEVELOPMENT TOOLS / TECHNOLOGY 7

5. STUDY OF EXISTING SYSTEM 9

6. FEASIBILITY 11

6.1 Technical Feasibility 11

6.2 Operational Feasibility 12

6.3 Economic Feasibility 13

7. REQUIREMENT ANALYSIS 14

7.1 Functional Requirements 14

7.2 Non-Functional Requirements 15

7.3 System Interface Requirements 16

7.4 Technical Constraints 17

8. SYSTEM DESIGN 18

8.1 Data Flow Diagram 18

8.2 ER Diagram 22

8.3 Functional Decomposition 24

8.4 Data Dictionary 26

9. DATABASE DESIGN 30

9.1 Database Structure 30

9.2 Normalization 32

10. CODING 34

10.1 Code Validations 34

10.2 Sample Code 38

11. IMPLEMENTATION 45

12. PLANNING SCHEDULE 50

13. TESTING 52

13.1 Test Strategies 52

13.2 Test Cases 54

14. FUTURE ENHANCEMENTS & CONCLUSION….....58

14.1 Conclusion 60

15. REFERENCES 62

1. ABSTRACT

The NGO Management Platform is a full-stack web-based application designed to help non-governmental organizations manage their activities more efficiently and improve communication with volunteers, donors, and the general public. The platform allows users to explore ongoing NGO projects, register as volunteers, read blogs and updates, view events, and contribute to social causes through donations.

The system serves three main types of users:

- **Admin** – Responsible for managing the overall platform, including projects, blogs, events, volunteers, and donation records. The admin also monitors platform activity and updates website content through an administrative dashboard.
- **Volunteers** – Individuals who register on the platform to participate in NGO activities and assist in different projects organized by the organization.
- **Users/Donors** – Visitors who can explore NGO initiatives, read blogs, view upcoming events, and support the organization through donations or by registering as volunteers.

The core functionalities of the platform include volunteer registration management, donation tracking, project and event display, blog publishing, and a centralized content management system that allows administrators to update website content dynamically. The platform also integrates a chatbot and contact form to assist users and handle common queries efficiently.

The system is developed using **React.js for the frontend, Node.js with Express.js for the backend, and MongoDB for database management**. RESTful APIs are used to enable secure communication between the client and server components, ensuring smooth data flow and efficient system performance.

By leveraging modern web technologies, the NGO Management Platform simplifies NGO operations, improves transparency in donation and volunteer management, and strengthens engagement between the organization and the community. The platform provides a user-friendly digital environment that helps NGOs manage their initiatives more effectively while encouraging public participation in social activities.

2. INTRODUCTION

The NGO Management Platform is a full-stack web-based application designed to improve the management and digital presence of non-governmental organizations. The platform allows users to explore NGO projects, read blogs and updates, register as volunteers, and support social initiatives through donations. It provides a centralized system that helps organizations manage their activities and interact with the public more effectively.

Traditional NGO operations often rely on manual record keeping and separate communication channels, which can lead to inefficiencies in managing volunteers, donations, and project information. The absence of a structured digital system makes it difficult for organizations to maintain transparency and keep the community informed about their activities.

This platform provides a centralized solution where administrators can manage projects, blogs, events, volunteers, and donations through a dedicated dashboard. The system also includes features such as a chatbot and contact forms to assist users and improve engagement.

By using modern web technologies, the platform offers a secure, scalable, and user-friendly environment that helps NGOs streamline their operations while increasing public participation in social initiatives.

2.1 Problem Definition

Non-Governmental Organizations often face challenges in managing volunteers, donations, and project information efficiently. Many NGOs still rely on manual record keeping, spreadsheets, and scattered communication methods, which can lead to inefficiencies in organizing activities and maintaining accurate records. As the number of volunteers and donors increases, managing this information without a proper system becomes difficult and time-consuming.

In addition, the absence of a centralized digital platform makes it challenging for NGOs to communicate updates about projects, events, and initiatives to the public.

Potential volunteers and donors may also find it difficult to learn about opportunities to contribute or participate in social initiatives.

The NGO Management Platform addresses these issues by providing a structured and user-friendly web system that allows organizations to manage their activities digitally. Through the platform, users can easily explore projects, register as volunteers, and make donations, while administrators can efficiently manage content, volunteers, and organizational activities through a dedicated dashboard.

2.2 Scope of the Project

The NGO Management Platform serves as a comprehensive digital system for managing various NGO activities and improving communication between the organization and the community. The platform provides essential functionalities such as project management, volunteer registration, donation tracking, blog publishing, and event management.

The system also includes role-based access where administrators can manage website content, monitor volunteer registrations, and track donations through an administrative dashboard. Public users can browse information about the NGO, view ongoing projects, read blogs, and participate in events or volunteer programs.

Additionally, the platform integrates interactive features such as a chatbot and contact forms to assist users and provide quick responses to common queries. By centralizing these functionalities within a single platform, the system improves transparency, simplifies management processes, and enhances engagement between NGOs and the public.

2.3 Objectives

The primary objectives of the NGO Management Platform are to:

- Provide a centralized digital platform for managing NGO activities and operations.
- Improve communication between NGOs, volunteers, and the general public.

- Simplify the process of volunteer registration and participation in NGO projects.
- Enable transparent management of donations and organizational activities.
- Provide an informative platform where users can explore projects, blogs, and events.
- Integrate interactive features such as chatbot support and contact forms for user assistance.
- Establish a secure and scalable system that supports efficient NGO management.

By achieving these objectives, the system improves organizational efficiency, encourages public participation in social initiatives, and helps NGOs manage their operations more effectively through a modern digital platform.

3. REQUIREMENT SPECIFICATION

3.1 Hardware Requirements

Component	Specification
Processor	Intel Core i3 (10th Generation) or higher
RAM	Minimum 4 GB (8 GB recommended)
System	Windows 10 / Linux (Ubuntu 20.04+) / macOS
Cache Size	3 MB L3 Cache or higher
Hard Disk Space	At least 500 MB free disk space
Internet Connection	Stable internet connection for accessing the web application

3.2 Software Requirements

Component	Specification
Operating System	Microsoft Windows / Linux / macOS
Programming Languages	JavaScript
Frontend Framework	React.js
Backend Framework	Node.js with Express.js
Database	MongoDB
Development Environment	Visual Studio Code
Version Control	Git and GitHub
Web Browser	Google Chrome / Mozilla Firefox / Microsoft Edge

4. DEVELOPMENT TOOLS / TECHNOLOGY

1. Integrated Development Environment (IDE)

- **Visual Studio Code (VS Code)**

  ❖ A lightweight and powerful code editor widely used for modern web development.

  ❖ Provides strong support for JavaScript, React.js, Node.js, and MongoDB development.

  ❖ Extensions such as Prettier, MongoDB for VS Code, and Thunder Client improve coding efficiency and debugging.

  ❖ **Purpose:** Used as the primary development environment for writing, editing, debugging, and managing both frontend and backend code.

2. Version Control & Collaboration

- **Git + GitHub**

  ❖ **Git:** A distributed version control system used to track code changes and manage development history.

  ❖ **GitHub:** A cloud-based repository hosting platform that allows developers to store, manage, and collaborate on project code.

  ❖ Supports version tracking, issue management, and team collaboration.

  ❖ **Purpose:** Helps maintain code integrity, manage project versions, and enable efficient collaboration during development.

3. Package Managers

- **npm (Node Package Manager)**

  ❖ npm is the default package manager for Node.js used to install and manage project dependencies.

  ❖ It allows developers to easily integrate third-party libraries and frameworks required for development.

❖ Packages such as Express.js, React libraries, Axios, and Mongoose are installed and maintained through npm.

❖ **Purpose:** Handles dependency management for both frontend and backend development.

4. Development Tools

- **Vite**

  ❖ A modern build tool used for faster frontend development with React.

  ❖ Provides fast startup time and efficient module bundling.

  ❖ Supports Hot Module Replacement (HMR), allowing instant updates during development.

  ❖ **Purpose:** Improves development speed and provides a smooth development workflow.

- **Tailwind CSS**

  ❖ A utility-first CSS framework used for designing responsive and modern user interfaces.

  ❖ Provides ready-to-use styling utilities that help build consistent and visually appealing layouts.

  ❖ **Purpose:** Used to design responsive UI components and improve the overall appearance of the website.

- **Prettier**

  ❖ An automated code formatting tool that ensures consistent coding style across the project.

  ❖ Helps maintain readable and well-structured code.

  ❖ **Purpose:** Improves code quality and maintains uniform formatting throughout the project.

5. Testing & API Validation

- **Postman**

  ❖ A popular API testing platform used to design, test, and debug REST APIs.

  ❖ Allows developers to send HTTP requests and analyze responses from backend services.

  ❖ **Purpose:** Used to test backend APIs such as authentication, donation handling, and volunteer registration endpoints.

- **Thunder Client**

  ❖ A lightweight API testing extension integrated directly into Visual Studio Code.

  ❖ Provides quick testing of API endpoints without switching to external applications.

  ❖ **Purpose:** Helps developers test APIs quickly during development.

6. Workflow & Optimization

- **Axios**

  ❖ A promise-based HTTP client used in React applications for sending requests to backend APIs.

  ❖ Enables communication between the frontend and backend services.

  ❖ **Purpose:** Used to fetch and submit data such as donations, volunteer registrations, and project details.

- **MongoDB Atlas**

  ❖ A cloud-based MongoDB database service used to store and manage application data.

  ❖ Provides secure and scalable data storage for user accounts, donations, volunteers, and projects.

  ❖ **Purpose:** Ensures reliable data storage and easy database management.

5. STUDY OF EXISTING SYSTEM

In today's digital era, many organizations and institutions use online platforms to manage their operations and communicate with users efficiently. Some NGOs have already adopted websites and digital tools that allow users to learn about their initiatives, donate to social causes, and participate in volunteer programs. However, many small and medium-sized NGOs still rely on manual systems or basic websites that provide only limited information about their activities.

Without a proper digital management system, NGOs often face challenges in organizing volunteer registrations, tracking donations, and sharing updates about ongoing projects. Important information about events, blogs, and social initiatives may not reach the intended audience effectively. This lack of a centralized platform can also reduce transparency and limit public engagement.

Furthermore, many existing NGO websites mainly provide static information and do not offer interactive features such as volunteer management systems, donation tracking, or content management tools. This makes it difficult for organizations to update information regularly or manage activities efficiently.

To address these limitations, the proposed **NGO Management Platform** has been designed to provide:

- A centralized platform where users can explore NGO projects, blogs, and upcoming events.
- A volunteer registration system that allows individuals to participate in social initiatives.
- A donation management system that helps track and manage contributions.
- An administrative dashboard that enables administrators to manage projects, volunteers, and website content efficiently.

By integrating these functionalities, the proposed system improves operational efficiency, increases transparency in NGO activities, and strengthens engagement between the organization and the community. This platform provides a structured and user-friendly approach to managing NGO operations and promoting social initiatives.

6. FEASIBILITY

Feasibility Criteria	Weightage (Percentage)
Operational Feasibility: The proposed NGO Management Platform provides significant benefits to the organization by streamlining the management of volunteers, donations, projects, and events through a centralized digital system. The platform improves communication between the NGO and the public by providing easy access to information about projects, blogs, and events. Administrators can efficiently manage website content, volunteer registrations, and donations through a dedicated dashboard. These features help reduce manual work and improve the overall efficiency of NGO operations.	**45%**
Technical Feasibility: The system is developed using modern web technologies such as React.js for the frontend, Node.js with Express.js for the backend, and MongoDB for database management. These technologies are widely used in modern web development and provide strong community support, reliability, and scalability. The platform also integrates additional features such as chatbot interaction and secure authentication. The availability of open-source frameworks and cloud-based deployment services ensures that the system can be implemented and maintained efficiently.	**35%**
Economic Feasibility: The development cost of the system is relatively low compared to the long-term benefits it provides to the organization. Most of the technologies used in the project are open-source, which significantly reduces software licensing costs. The primary expenses involved include domain registration, hosting services, and basic server infrastructure. Considering the benefits of improved transparency, better volunteer management, and efficient donation tracking, the system is a cost-effective solution for NGOs.	**20%**

7. REQUIREMENT ANALYSIS

1. Functional Requirements

User & Admin Management

- Only administrators can log in to the system using secure authentication credentials.
- Admins can manage website content including projects, blogs, events, and volunteer data.
- Admins can review and manage volunteer registrations submitted through the platform.
- The system restricts administrative features to authorized users only.
- User and volunteer information is stored securely in the database for management purposes.

Project Management

- Users can browse information about NGO projects without logging in.
- Projects are categorized and displayed clearly for easy navigation.
- Each project contains details such as description, objectives, and images.
- Admins can add, update, or delete project information through the administrative dashboard.

Volunteer Management

- Individuals can register as volunteers through an online registration form.
- Admins can review volunteer applications and manage volunteer data.
- Approved volunteers can participate in NGO activities and initiatives.
- Volunteer records are stored and maintained for future coordination.

Donation Management

- Users can support NGO initiatives by making donations through the platform.
- Donation details are recorded and stored in the system.
- Admins can monitor and manage donation records.
- The platform promotes transparency in the handling of contributions.

Blog & Event Management

- Users can read blogs and updates related to NGO activities.
- The platform displays upcoming events organized by the NGO.
- Admins can create, edit, or delete blog posts and event announcements.
- Content updates can be performed through the admin dashboard.

Chatbot Integration

- Users can interact with a chatbot for basic queries related to NGO activities.
- The chatbot helps users navigate the platform and obtain quick information.
- It assists users with volunteer registration and donation-related inquiries.

Communication & Contact

- Users can contact the organization through a contact form on the website.
- Admins can view and respond to user inquiries.
- The system helps maintain communication between the NGO and the community.

Secure Data Management

- User, volunteer, and donation information is stored securely in the database.
- Only administrators have login credentials to manage system data.
- Data validation ensures that only valid information is stored in the system.

2. Non-Functional Requirements

Performance & Scalability

- The system should support multiple users browsing projects and registering as volunteers simultaneously.
- The backend should be scalable to support future expansion of NGO activities and users.

Security

- Only authorized administrators can access the admin dashboard.
- Role-based access control ensures only permitted users can modify system data.

Availability & Reliability

- The system should be accessible to users at all times with minimal downtime.
- Data backup mechanisms should be implemented to prevent data loss.

Usability & Accessibility

- The user interface should be simple, intuitive, and easy to navigate.
- The system should be mobile-responsive for access across various devices.

Maintainability & Modularity

- The system should follow a modular architecture for easy updates and maintenance.
- The codebase should allow developers to add new features without affecting existing functionality.

Integration & Compatibility

- The system should be compatible with modern web browsers and operating systems.
- It should support integration with external services such as chatbot systems and hosting platforms.

3. System Interface Requirements

User Interface (UI)

- A responsive web-based interface developed using React.js.
- Interactive pages displaying NGO projects, blogs, events, and volunteer registration forms.
- An administrative dashboard for managing website content and organizational data.
- A chatbot interface that assists users with queries and navigation.

External Interfaces

- **MongoDB Database:** Used for storing user, volunteer, project, blog, and donation data.

- **RESTful API (Node.js + Express.js):** Handles communication between frontend and backend components.
- **Chatbot Integration:** Provides automated responses for common user queries.

Hardware & Software Interfaces

- **Frontend:** Built with React.js and styled using Tailwind CSS for responsive design.
- **Backend:** Developed with Node.js and Express.js to handle API requests and business logic.
- **Database:** MongoDB used for storing and managing application data.

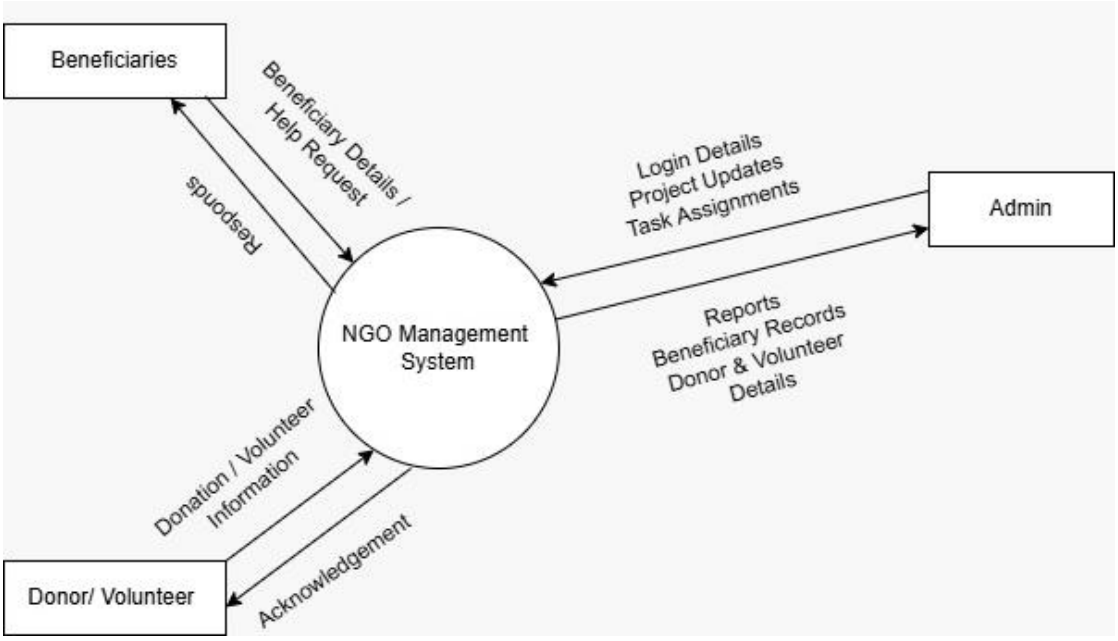
4. Technical Constraints

- The system must support multiple users accessing the platform simultaneously without performance issues.
- Internet connectivity is required for users to access the web-based application.
- Data storage capacity depends on the database hosting plan and may require upgrades in the future.
- The chatbot operates based on predefined responses and may have limitations in handling complex queries.
- Hosting, domain registration, and server infrastructure may involve additional operational costs.

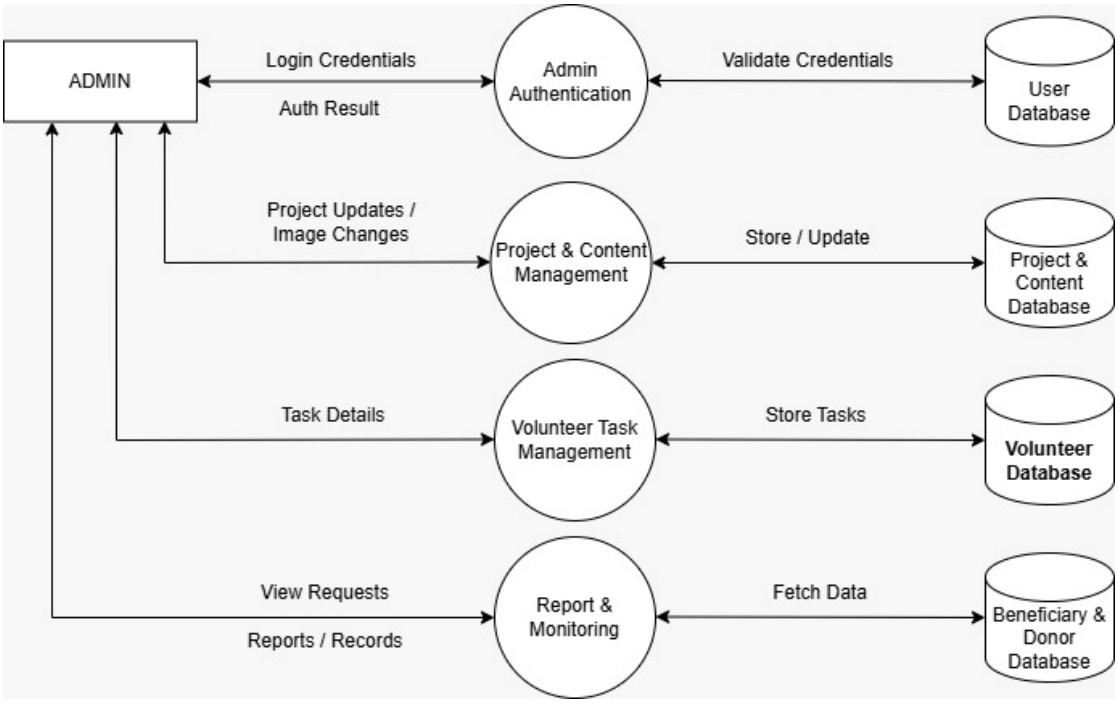
8. System Design

8.1 Data Flow Diagram

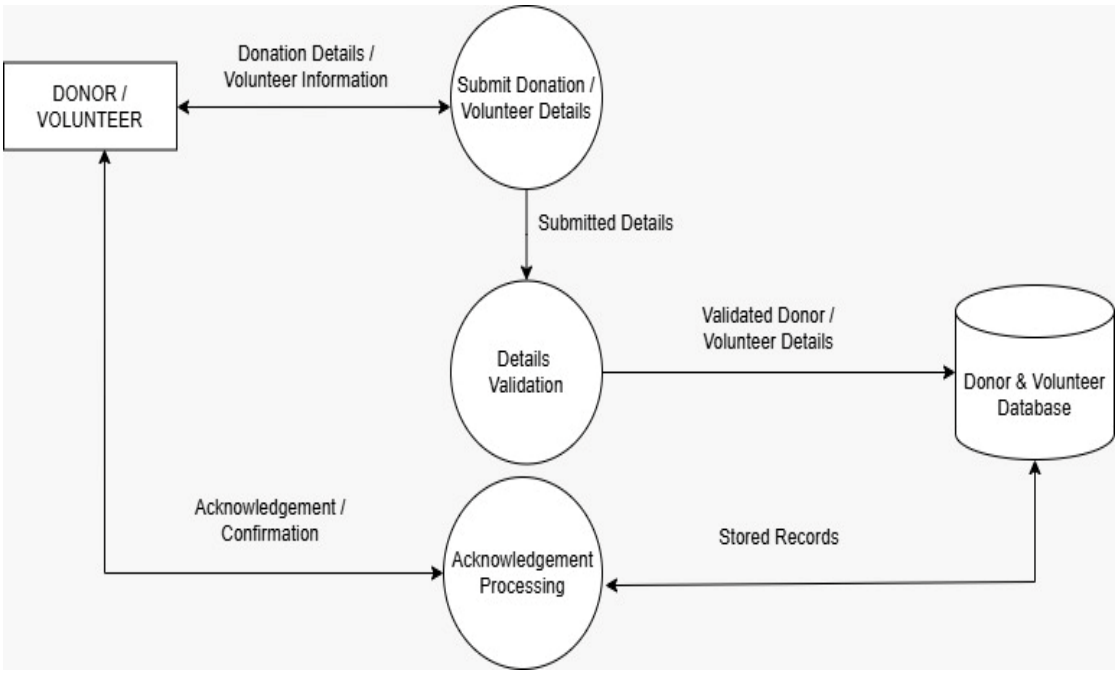
Level 0 :



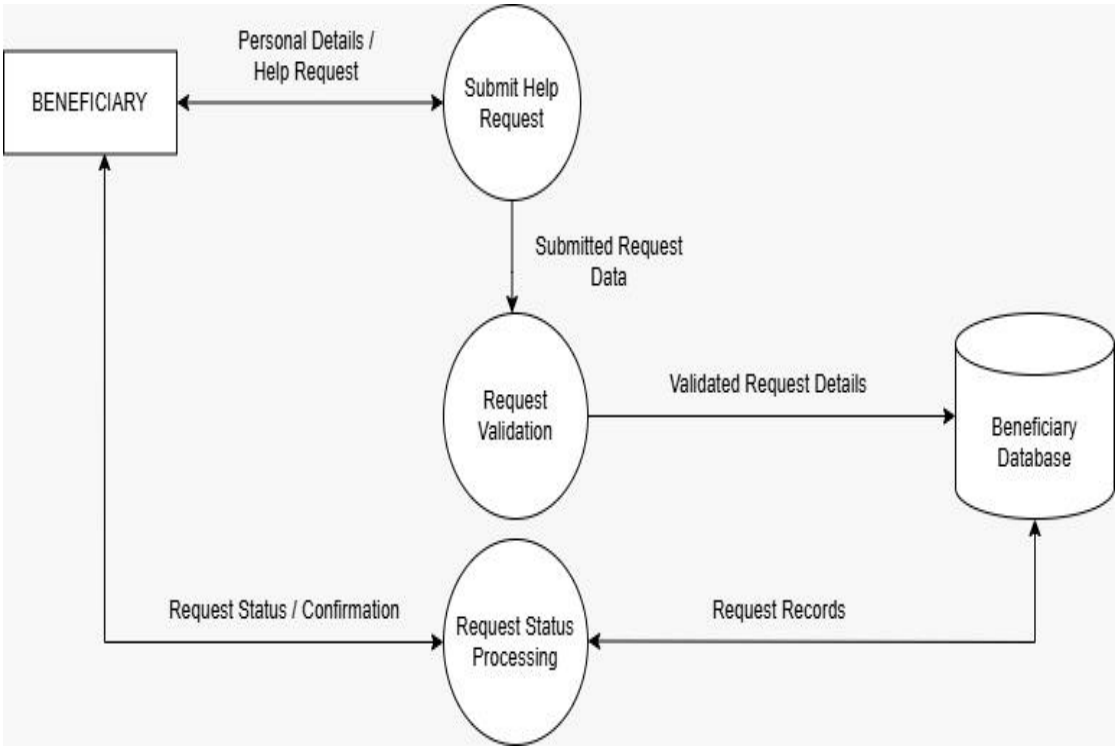
Level 1 : (Admin)



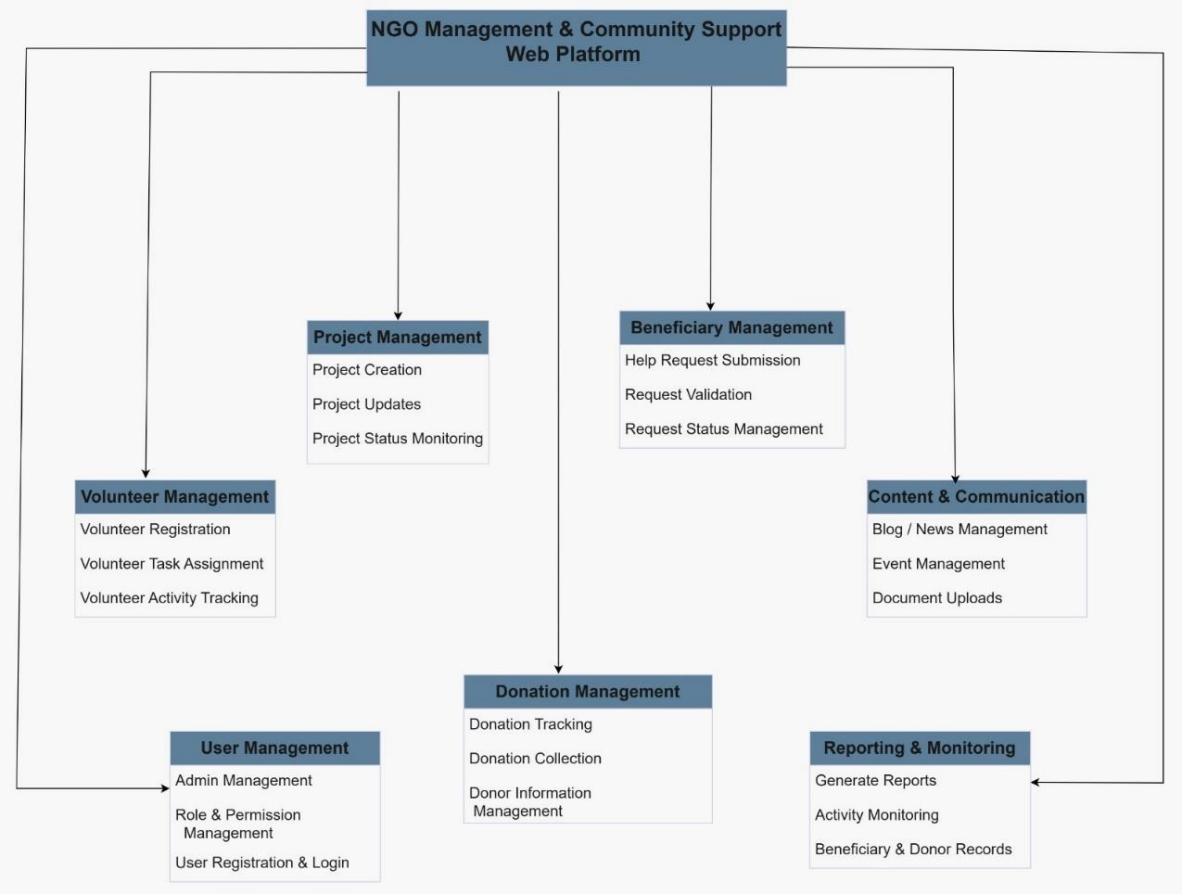
Level 1 : (Donor and Volunteer)



Level 1 : (Beneficiary)



8.2 Functional Decomposition



8.4 Data Dictionary

Collections:

1. Users

Field Name	Data Type	Field Length	Constraints	Description
id	ObjectId	–	Auto-generated, Unique	Unique identifier for each user
name	String	Variable	Required	Name of the user
email	String	Variable	Required, Unique	Email used for login
role	String	Variable	Required	Role of the user (Admin / Volunteer / Donor)
passwordHash	String	Variable	Required	Hashed password for security
createdAt	Date	–	Auto-generated	Account creation timestamp

2. Volunteers

Field Name	Data Type	Field Length	Constraints	Description
id	ObjectId	–	Auto-generated, Unique	Unique identifier for each volunteer
userId	ObjectId	–	Reference User	Linked user account
status	String	Variable	Required	Volunteer availability status
task	Array	–	References VolunteerTask	Tasks assigned to volunteer

3. Projects

Field Name	Data Type	Field Length	Constraints	Description
id	ObjectId	–	Auto-generated	Unique identifier for project
title	String	Variable	Required	Project title
description	String	Variable	Required	Details of the project
status	String	Variable	Required	Current project status
createdBy	ObjectId	–	Reference User	Admin who created project

4. VolunteerTasks

Field Name	Data Type	Field Length	Constraints	Description
id	ObjectId	–	Auto-generated	Unique task ID
title	String	Variable	Required	Task name
description	String	Variable	Optional	Task details
volunteerId	ObjectId	–	Reference Volunteer	Assigned volunteer
projectId	ObjectId	–	Reference Project	Associated project

5. Donations

Field Name	Data Type	Field Length	Constraints	Description
id	ObjectId	–	Auto-generated	Unique donation ID
name	String	Variable	Required	Donor name
amount	Number	–	Required	Donation amount
date	Date	–	Auto-generated	Date of donation

6. Blogs

Field Name	Data Type	Field Length	Constraints	Description
id	ObjectId	–	Auto-generated	Blog identifier
title	String	Variable	Required	Blog title
content	String	Variable	Required	Blog content
authorId	ObjectId	–	Reference User	Author of the blog

7. Events

Field Name	Data Type	Field Length	Constraints	Description
id	ObjectId	–	Auto-generated	Event identifier
title	String	Variable	Required	Event name
date	Date	–	Required	Event date
description	String	Variable	Optional	Event details

8. ActivityLogs

Field Name	Data Type	Field Length	Constraints	Description
id	ObjectId	–	Auto-generated	Activity log ID
action	String	Variable	Required	Action performed
userId	ObjectId	–	Reference User	User who performed action
timestamp	Date	–	Auto-generated	Time of action

9. Documents

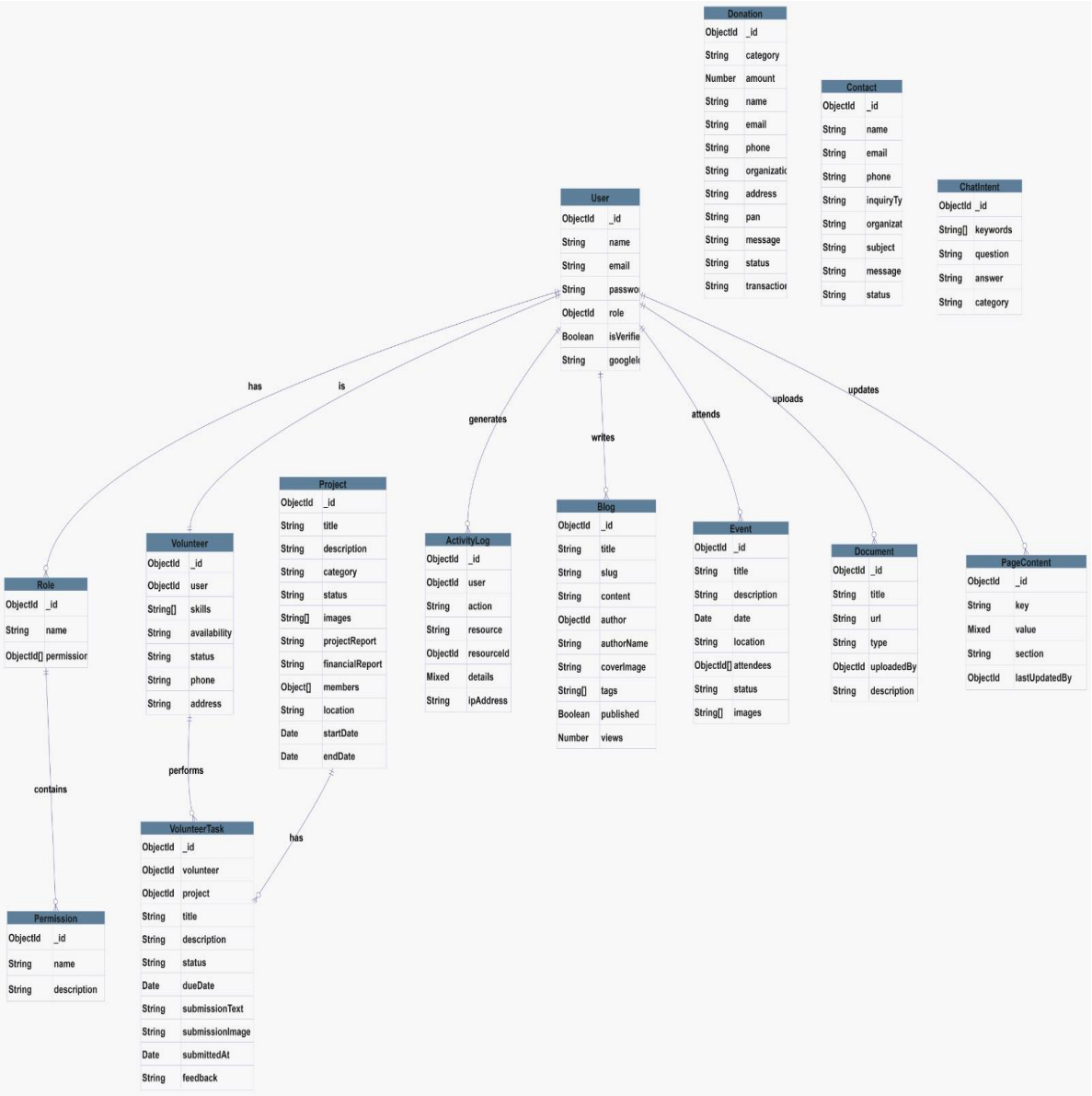
Field Name	Data Type	Field Length	Constraints	Description
id	ObjectId	–	Auto-generated	Document ID
title	String	Variable	Required	Document title
fileUrl	String	Variable	Required	Stored file location
uploadedBy	ObjectId	–	Reference User	User who uploaded document

10. ChatIntents

Field Name	Data Type	Field Length	Constraints	Description
id	ObjectId	–	Auto-generated	Chat intent ID
question	String	Variable	Required	Question asked
response	String	Variable	Required	Bot response

9. DATABASE DESIGN

9.1 Database Structure



10. CODING

10.1 Code Validations

Frontend Code Validation

To improve user experience and reduce errors before data is sent to the backend, the frontend implements several validation mechanisms within React components. These validations ensure that the data entered by users is accurate and complete before submission.

Required Field Validation

Key fields across various forms such as **Volunteer Registration Form, Donation Form, and Contact Form** are marked as required.
This prevents users from submitting incomplete forms and ensures that essential information such as name, email, phone number, and message details are provided before submission.

- **Email Validation**
- **Input Pattern Validation:** Email fields use standard email validation patterns to ensure correct formatting.
- **Real-time Feedback:** The form displays error messages when an invalid email format is entered.
- **Purpose:** Prevents incorrect email addresses from being submitted.
- **Phone Number Validation**
- **Numeric Input Restriction:** Phone number fields restrict input to numeric characters only.
  - **Regex Pattern Validation:** A regular expression pattern (e.g., pattern="\d{10}") ensures the phone number contains exactly ten digits.
  - **Input Sanitization:** JavaScript event handlers remove non-numeric characters and limit the input length to prevent invalid entries.

Form Submission Validation

Before submitting any form, the system checks whether all required fields are filled correctly. If any required data is missing or invalid, the form submission is blocked and an error message is displayed.

Terms & Conditions Acceptance

Certain forms, such as volunteer registration or donation forms, include a mandatory checkbox requiring users to agree to the **Terms and Conditions** before submission. The form cannot be submitted until the checkbox is selected.

Real-Time Feedback

When a user submits a form, the system provides immediate feedback depending on the backend response.

● **Success messages** confirm that the form has been successfully submitted.
● **Error messages** guide users to correct missing or invalid input fields.

Login Validation and Protected Routes

The admin login page ensures that only authorized administrators can access the dashboard.

● Login forms require both **email and password** fields to be filled.
● The system verifies authentication using a secure token system.
● **Protected Routes:** Certain pages such as the admin dashboard are accessible only after successful authentication.
● Unauthorized users attempting to access restricted pages are redirected to the login page.

Backend Code Validation and Security

To maintain system security and ensure data consistency, the backend enforces multiple validation mechanisms.

Schema Validation

MongoDB schemas defined using **Mongoose** enforce the structure of the data stored in the database.

● Each schema defines **required fields, data types, and validation rules**.
● Models such as **User, Volunteer, Donation, Blog, and Project** follow strict schema validation to maintain data integrity.
● Default values and constraints ensure structured and consistent data storage.

Input Sanitization & Error Handling

Backend controllers validate incoming request data before processing.

● Missing or invalid fields are rejected with appropriate error responses.
● Server-side validation ensures that only valid data is inserted into the database.
● Error-handling middleware captures unexpected errors and returns appropriate responses to the client.

Authentication & Authorization

The system uses secure authentication mechanisms to protect administrative functions.

● Authentication verifies the identity of users attempting to access the system.
● Authorization ensures that only administrators can access management features such as editing projects, blogs, or volunteer data.
● Middleware functions verify authentication tokens before allowing access to protected routes.

API Request Validation

All API endpoints validate incoming request parameters before processing requests.

● Each API route checks the presence and format of required data fields.
● Example: Volunteer registration endpoints validate name, contact information, and form data before saving records.

Endpoint Security

Sensitive administrative operations are protected through access control mechanisms.

● Only administrators can create, update, or delete projects, blogs, and events.
● Volunteer and donation records can only be accessed or modified by authorized users.
● Role-based access control prevents unauthorized users from modifying system data.

These validation measures help maintain **data accuracy, application stability, and system security**, ensuring that the NGO Management Platform operates reliably for both administrators and public users.

4. Sample Code

This section comprehensively details the purpose, mechanisms, and specific responsibilities of each directory and file utilized across both the frontend React application and backend Node/Express server.

4.1 Frontend (frontend/src)

The frontend is a Single Page Application (SPA) built using React, bundled with Vite, and styled with TailwindCSS. It handles routing on the client-side via React Router.

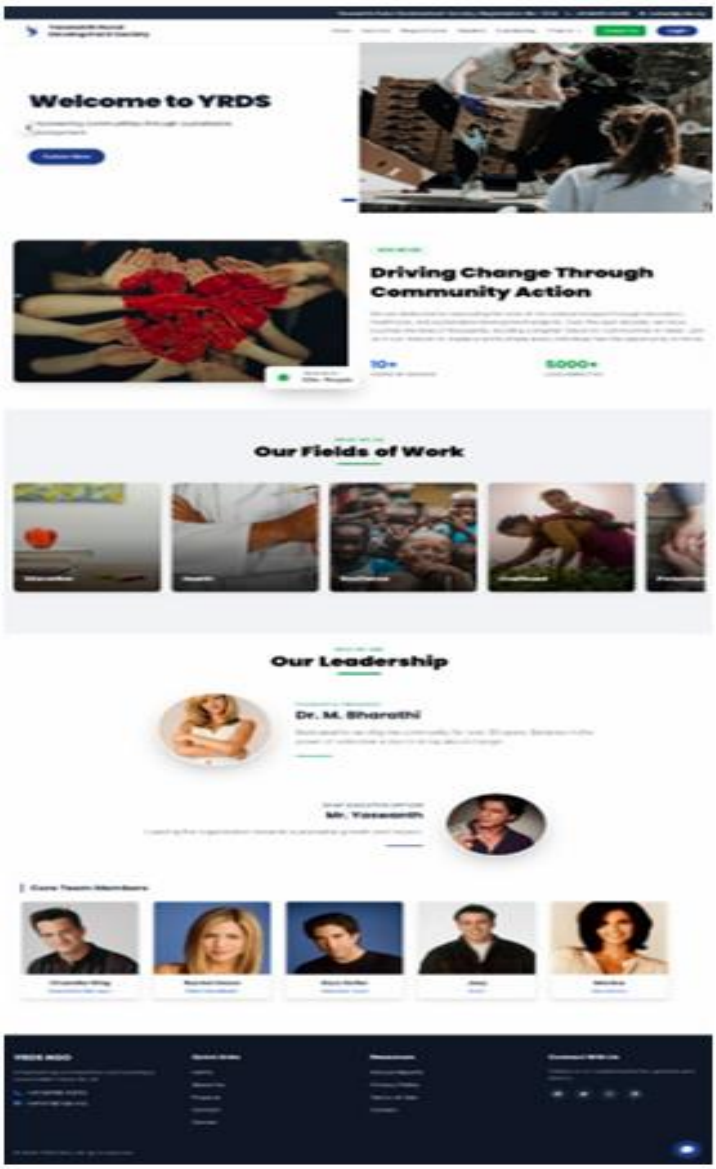
Pages (src/pages)

These are the primary top-level container components displayed by React Router. Each corresponds to a major path in the application and is responsible for fetching initial data, managing page-level state, and composing smaller UI components.

- **Home.jsx (/)**: The main landing page. It acts as a marketing dashboard, aggregating distinct UI sections (carousels, mission summaries, impact metrics, and team member highlights) to present a high-level overview of the NGO, serving as the first touchpoint for public users.

```
import React from 'react';
import CarouselSection from '../components/home/CarouselSection';
import SummarySection from '../components/home/SummarySection';
import FieldsOfWork from '../components/home/FieldsOfWork';
import MembersSection from '../components/home/MembersSection';

const Home = () => {
    return (
        <div className="homepage">
            <CarouselSection />
            <SummarySection />
            <FieldsOfWork />
            <MembersSection />
        </div>
    );
};
export default Home;
```

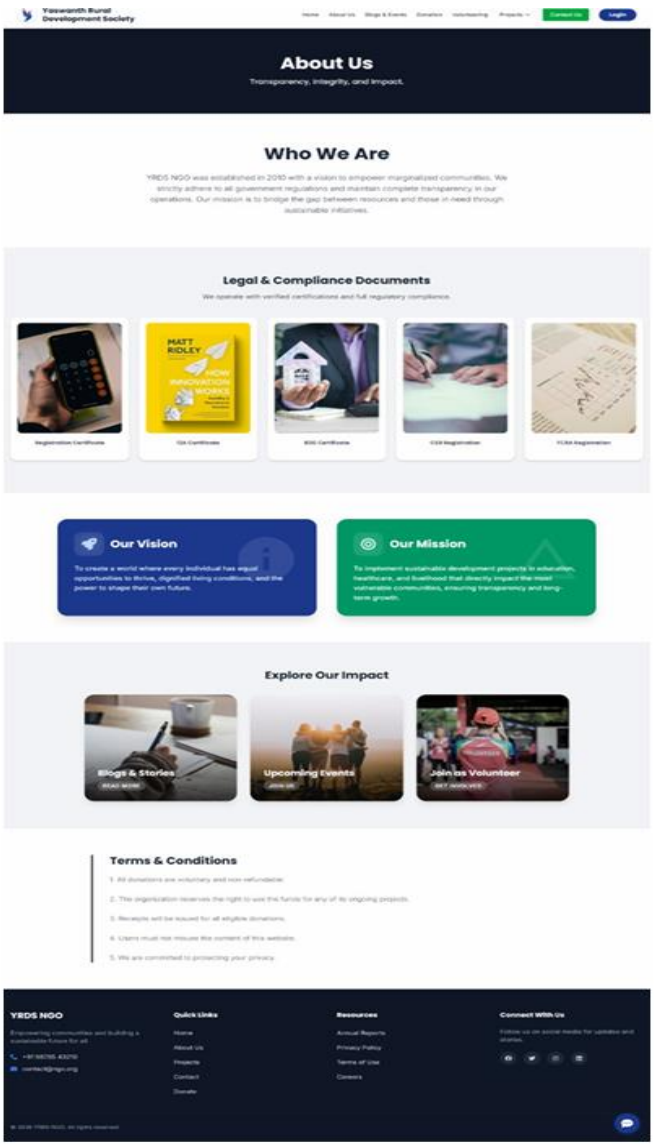
Home page outcome



- **About.jsx (/about)**: Presents detailed narrative information outlining the NGO's core mission, historical timeline, values, and explicitly introducing core team members to build trust.

```
import React from 'react';
import EditableText from '../components/cms/EditableText';
import DynamicList from '../components/cms/DynamicList';
import EditableImage from '../components/cms/EditableImage';

const About = () => {
    return (
        <div className="min-h-screen font-sans">
            <div className="bg-gray-900 py-20 text-center text-white">
                <h1 className="text-4xl md:text-5xl font-bold mb-4">
                    <EditableText contentKey="about_header_title" defaultText="About Us" />
                </h1>
            </div>
        </div>
    );
}
```

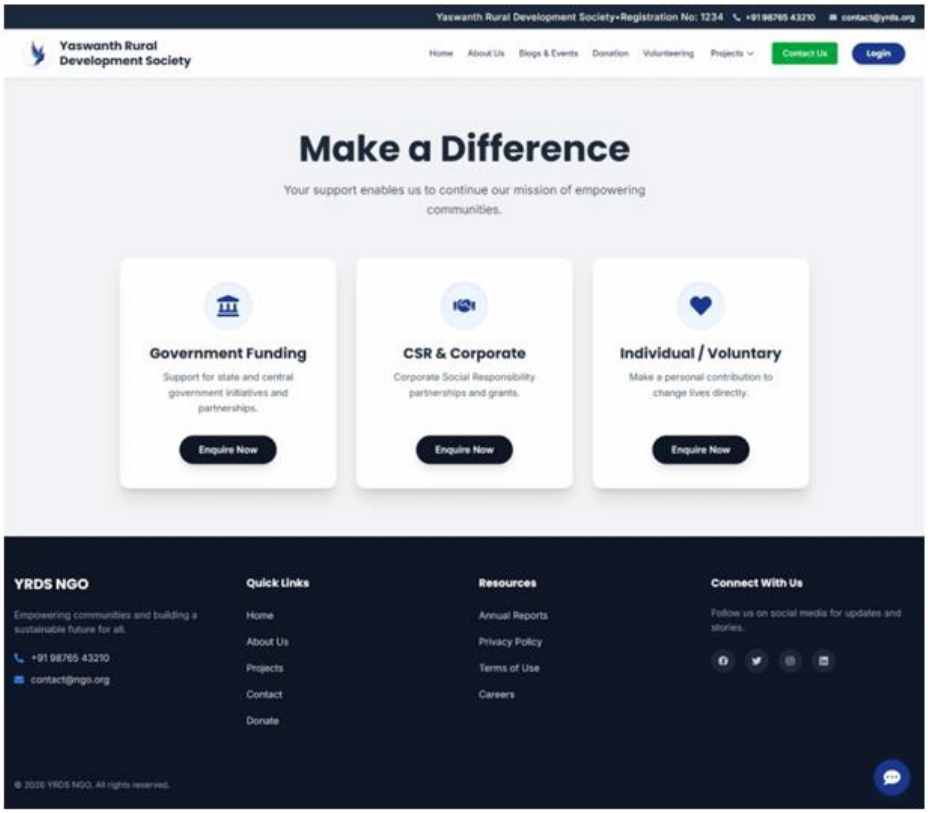
About page outcome



- **Projects.jsx (/projects)**: Displays a searchable, filterable catalog of ongoing and completed initiatives/campaigns. It pulls dynamic project data from the backend to provide transparency into the NGO's actual on-the-ground work.
- **Donate.jsx (/donate)**: The financial portal. It handles the user interface and frontend logic specifically built for gathering donor information, selecting donation amounts, and integrating with payment gateways to process monetary contributions securely.

```
const mongoose = require('mongoose');

const donationSchema = new mongoose.Schema({
    category: {
        type: String,
        enum: ['Government', 'Corporate', 'Voluntary'],
        required: true
    },
    amount: {
        type: Number,
        required: false
    },
    // Donor Details
    name: {
        type: String,
        required: true
    },
    email: {
        type: String,
        required: true
    },
    phone: {
        type: String
    },
    organization: { // For Government/Corporate
        type: String
    },
    address: {
        type: String
    },
    pan: { // Tax requirement
        type: String
```

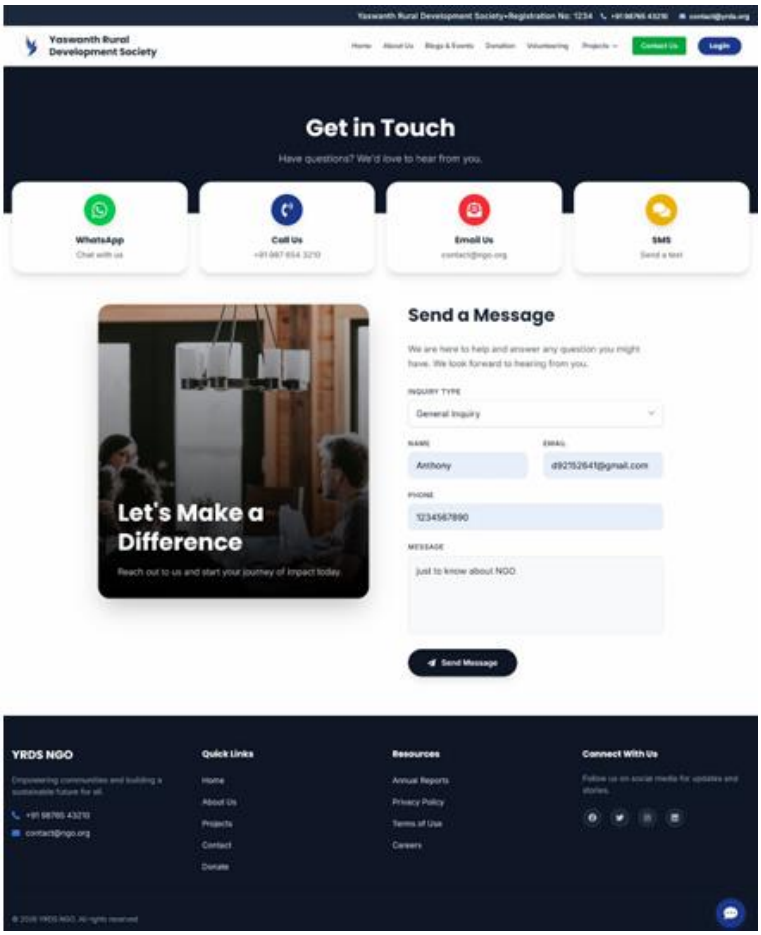
Donation page outcome



- **Contact.jsx (/contact)**: Renders a user-friendly form allowing visitors to send direct inquiries to the NGO. It also explicitly lists alternative communication channels like interactive WhatsApp integration, direct Email links, and Phone numbers.

```jsx
import React, { useState } from 'react';
import { useAuth } from '../context/AuthContext';
import { useNavigate } from 'react-router-dom';

const Login = () => {
    const [isLogin, setIsLogin] = useState(true);
    const [formData, setFormData] = useState({ email: '', password: '' });
    const { login, register } = useAuth();
    const navigate = useNavigate();
// ...
```

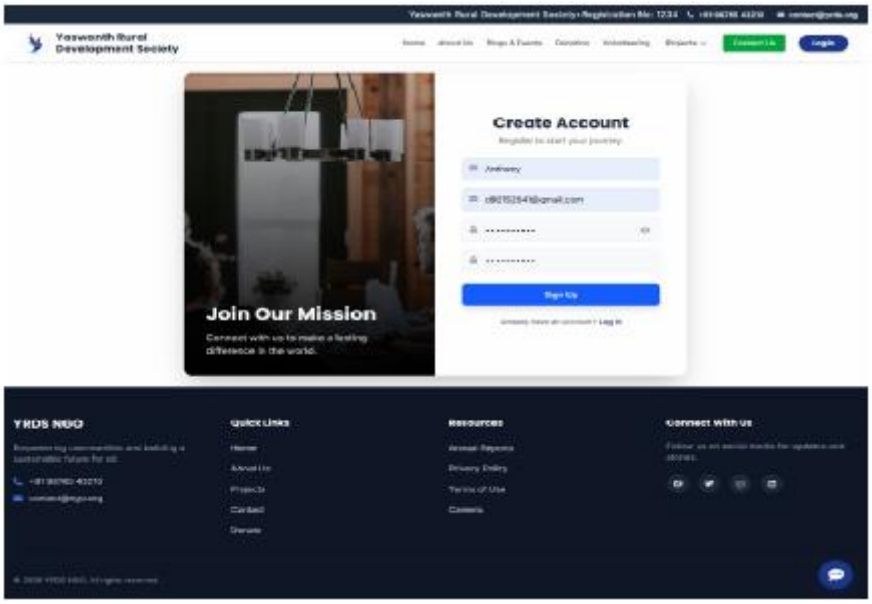
Contact page outcome



- **Login.jsx (/login)**: A dual-purpose authentication screen handling both "Sign Up" (registration) and "Sign In" (authentication) flows. It seamlessly toggles between the two modes, capturing credentials and interfacing with the AuthContext.

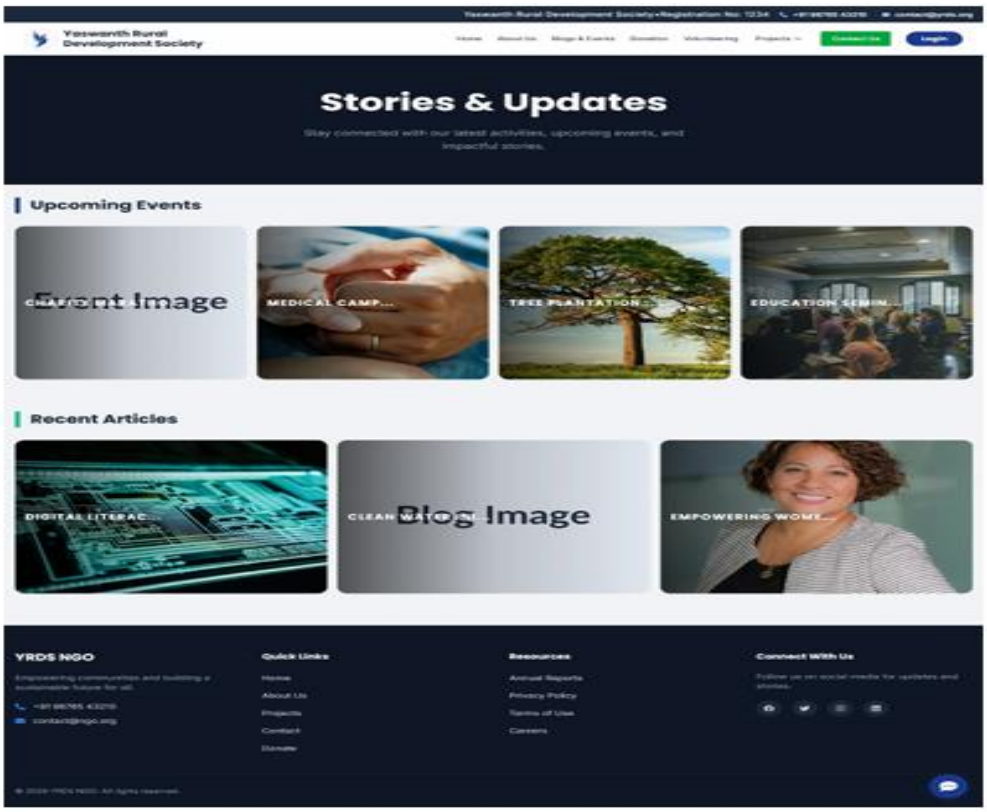
```jsx
import React, { useState } from 'react';
import { useAuth } from '../context/AuthContext';
import { useNavigate } from 'react-router-dom';

const Login = () => {
    const [isLogin, setIsLogin] = useState(true);
    const [formData, setFormData] = useState({ email: '', password: '' });
    const { login, register } = useAuth();
    const navigate = useNavigate();
//
```

Login page outcome



- **BlogsEvents.jsx (/media or /news)**: Combines recent blog articles and upcoming event schedules into a single media consumption interface. It fetches arrays of media data from the backend to display news feeds and chronological event cards.

Blogs outcome

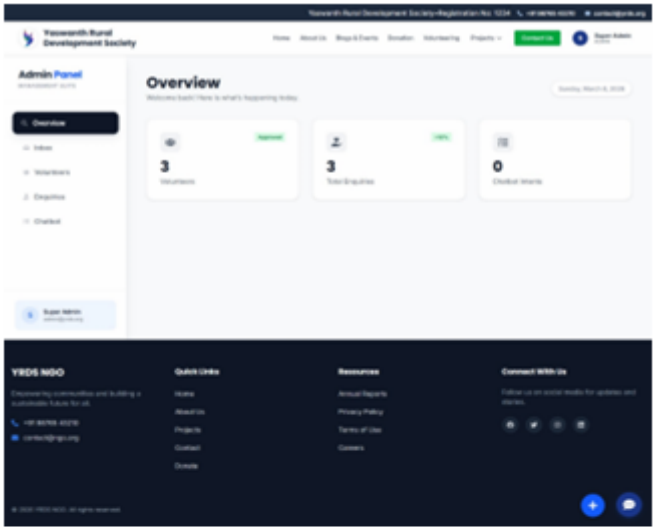


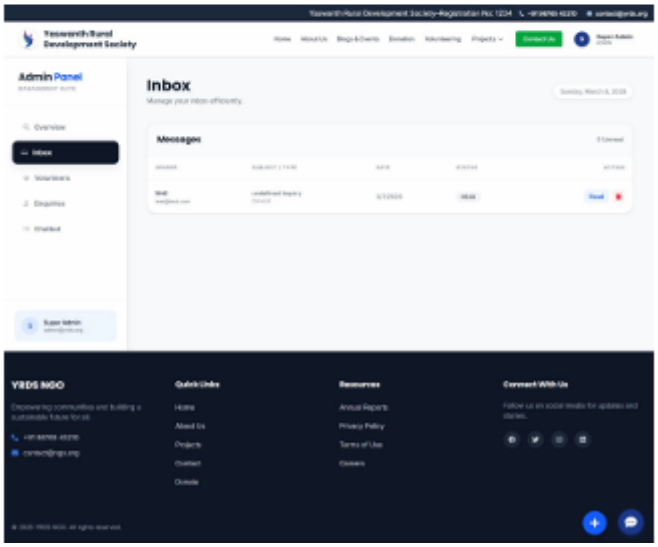
```
const Blog = require('../models/Blog');
const Event = require('../models/Event');

// @desc    Get Published Blogs
// @route   GET /api/media/blogs
// @access  Public
const getBlogs = async (req, res) => {
    try {
        const blogs = await Blog.find({ published: true }).sort({ created/
        res.json(blogs);
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
};

// @desc    Get All Events
// @route   GET /api/media/events
// @access  Public
const getEvents = async (req, res) => {
    try {
        const events = await Event.find().sort({ date: 1 });
        res.json(events);
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
};

// @desc    Create a Blog
// @route   POST /api/media/blogs
// @access  Private/Admin
const createBlog = async (req, res) => {
```

```
const createBlog = async (req, res) => {
Click to collapse the range.
        const blog = await Blog.create(req.body);
        res.status(201).json(blog);
    } catch (error) {
        res.status(400).json({ message: error.message });
    }
};

// @desc    Update a Blog
// @route   PUT /api/media/blogs/:id
// @access  Private/Admin
const updateBlog = async (req, res) => {
    try {
        const blog = await Blog.findByIdAndUpdate(req.params.id, req.body,
        if (!blog) return res.status(404).json({ message: 'Blog not found'
        res.json(blog);
    } catch (error) {
        res.status(400).json({ message: error.message });
    }
};

// @desc    Delete a Blog
// @route   DELETE /api/media/blogs/:id
// @access  Private/Admin
const deleteBlog = async (req, res) => {
    try {
        const blog = await Blog.findByIdAndDelete(req.params.id);
        if (!blog) return res.status(404).json({ message: 'Blog not found'
        res.json({ message: 'Blog removed' });
    } catch (error) {
```

- **Dashboard.jsx (/admin)**: A highly secure, protected route acting as the central administrative hub. It allows authorized personnel (Admins) to manage users, update CMS content across the site, review incoming donations, track active projects, and administer volunteer approvals.

```jsx
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useAuth } from '../context/AuthContext';
import { useCMS } from '../context/CMSContext';

const Dashboard = () => {
    const { user } = useAuth();
    const isAdmin = user?.role?.name === 'Admin';
    const [activeTab, setActiveTab] = useState('Overview');
    // ...
```

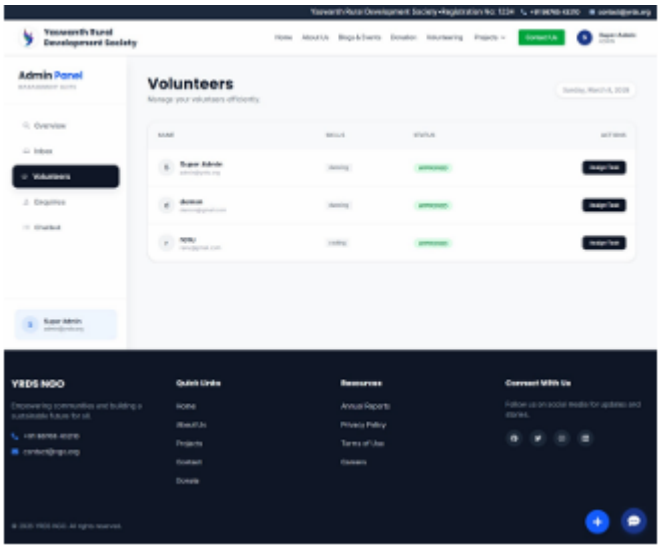


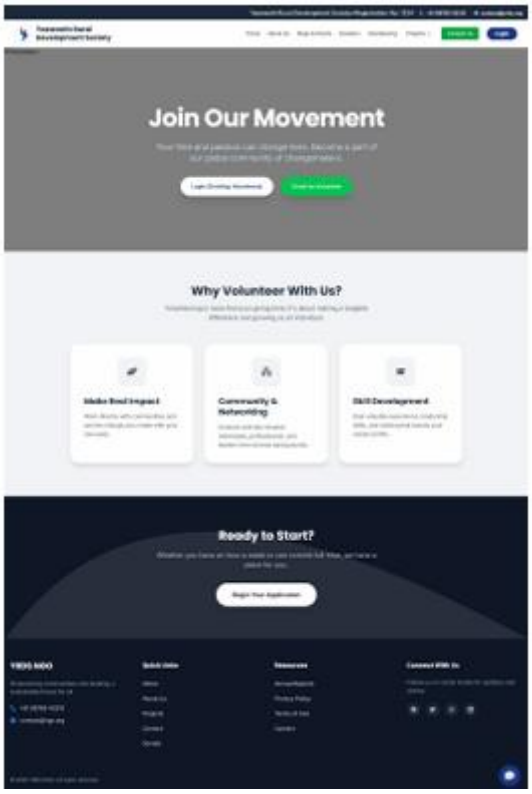


- **Volunteer/VolunteerLanding.jsx (/volunteer)**: An informational marketing page specifically designed to encourage users to sign up for volunteering efforts, explaining the benefits and impact they can have.
- **Volunteer/VolunteerRegister.jsx (/volunteer/register)**: A detailed, specialized application form requesting users to become certified volunteers. It gathers critical vetting data such as specific skills, geographical location, and availability schedules.
- **Volunteer/VolunteerDashboard.jsx (/volunteer/dashboard)**: A specialized, protected interface limited strictly to approved volunteers. It allows them to view their assigned operational tasks, update task statuses, and view internal announcements.

```
const Volunteer = require('../models/Volunteer');
const VolunteerTask = require('../models/VolunteerTask');
const User = require('../models/User');
const Role = require('../models/Role');

// @desc    Register as Volunteer
// @route   POST /api/volunteers/register
// @access  Private (Any User)
const registerVolunteer = async (req, res) => {
    try {
        const { skills, availability, phone, address } = req.body;

        // Check if already registered
        const existing = await Volunteer.findOne({ user: req.user._id });
        if (existing) {
            return res.status(400).json({ message: 'You have already regis
        }

        const volunteer = await Volunteer.create({
            user: req.user._id,
            skills,
            availability,
            phone,
            address,
            status: 'Pending'
        });

        // Assign Role 'Volunteer' to User
        let volunteerRole = await Role.findOne({ name: 'Volunteer' });

        if (!volunteerRole) {
            volunteerRole = await Role.create({ name: 'Volunteer', permiss
```

Volunteers Dashboard and other outcomes





Components (src/components)

These are highly reusable, granular React components segmented into logical domain groups. They rely on "Pages" to pass down data via props, keeping them "dumb" and purely focused on presenting UI.

- **layout/Header.jsx**: The universal primary navigation bar present at the top of every screen. It contains responsive hamburger menus for mobile and dynamic link rendering (e.g., showing a "Logout" button only if AuthContext detects a user).
- **layout/Footer.jsx**: The universal page footer containing quick links, copyright info, newsletter signups, and social media connectivity buttons.
- **home/CarouselSection.jsx**: Displays dynamic, sliding hero images on the landing page for high visual impact, often linked to current campaigns.
- **home/FieldsOfWork.jsx**: Highlights specific functional domains the NGO focuses on (e.g., Education, Healthcare, Disaster Relief) mapped out in an interactive grid or card layout.

- **home/MembersSection.jsx**: Shows key team members, their roles, and short biographies on the landing page.
- **home/SummarySection.jsx**: Provides a quick, scannable summary metric (e.g., "10,000 Lives Impacted") or a bold mission statement block for immediate context.
- **home/TopMarquee.jsx**: A scrolling banner component used at the very top of the screen to flag urgent announcements, critical appeals, or live event updates globally.
- **cms/DynamicList.jsx**: An interactive, dynamic list element utilized heavily in the admin dashboard. It allows dragging, dropping, or sequencing data arrays visually before saving changes to the backend.
- **cms/EditToggle.jsx**: A specialized, sticky switch providing admins the ability to quickly enter a global "edit mode" across the site, altering how standard components render.
- **cms/EditableImage.jsx**: A sophisticated wrapper component. When "edit mode" is active, it overlays an upload button on standard website images, allowing admins to seamlessly swap out visual assets directly from the frontend interface.
- **cms/EditableText.jsx**: A text component that seamlessly converts into a content-editable div or <textarea> when "edit mode" is active. It captures keystrokes and dispatches API calls to update the PageContent database via the CMS.
- **common/ImageWithFallback.jsx**: A highly resilient image wrapper element. If an intended image URL fails to load (404), this component intercepts the error and substitutes a sleek default silhouette or organizational pattern to prevent broken UI layouts.
- **common/SelectInput.jsx**: A unified, styled dropdown .select field component, ensuring consistent aesthetics and behavior across all forms in the application.

- **chat/ChatWidget.jsx**: A floating, interactive chat bubble interface. It supplies live or automated chat support to end-users on any page by conversing with the backend's intent-matching AI/logic.

Context (src/context)

Components managing complex, globally shared state using the React Context API. This prevents "prop-drilling" down dozens of component layers.

- **AuthContext.jsx**: The absolute core of frontend security. It centralizes user session state, managing the exact presence of the JWT in local storage. It exposes standard login, logout, and register functions globally, and maintains the current user's role profile to dictate conditional UI rendering (e.g., hiding the Admin button from standard users).
- **CMSContext.jsx**: Centralizes the global "edit mode" boolean state. It broadcasts to arbitrary UI elements (like EditableText) whether they should render as normal text or as an interactive admin input field.

Utilities (src/utils)

Helper functions and High-Order Components (HOCs) used to abstract complex, repetitive logic.

- **PrivateRoute.jsx**: A High-Order Component wrapping sensitive routes. It intercepts navigation attempts, checks the AuthContext, and violently redirects unauthenticated users back to the /login screen.
- **RoleRoute.jsx**: An advanced High-Order Component guarding highly sensitive routes. Not only does it check for authentication, but it also parses the user's explicit Role (e.g., stopping a standard user from accessing /admin, or stopping an Admin from accessing a Volunteer-only /volunteer/dashboard).
- **imageUrl.js**: A pure functional utility responsible for formatting and normalizing relative paths for uploaded images returned from the backend

(e.g., /uploads/image.jpg) into absolute, fully resolved network paths the browser can render.

4.2 Backend (backend/)

The backend is a robust RESTful API built on Node.js using the Express framework. It connects to a MongoDB NoSQL database via Mongoose.

Models (backend/models)

Mongoose schemas functioning as the sole, rigid source of truth for MongoDB database structures. They enforce strict data types, relational bindings, and unique constraints before any data is written to disk.

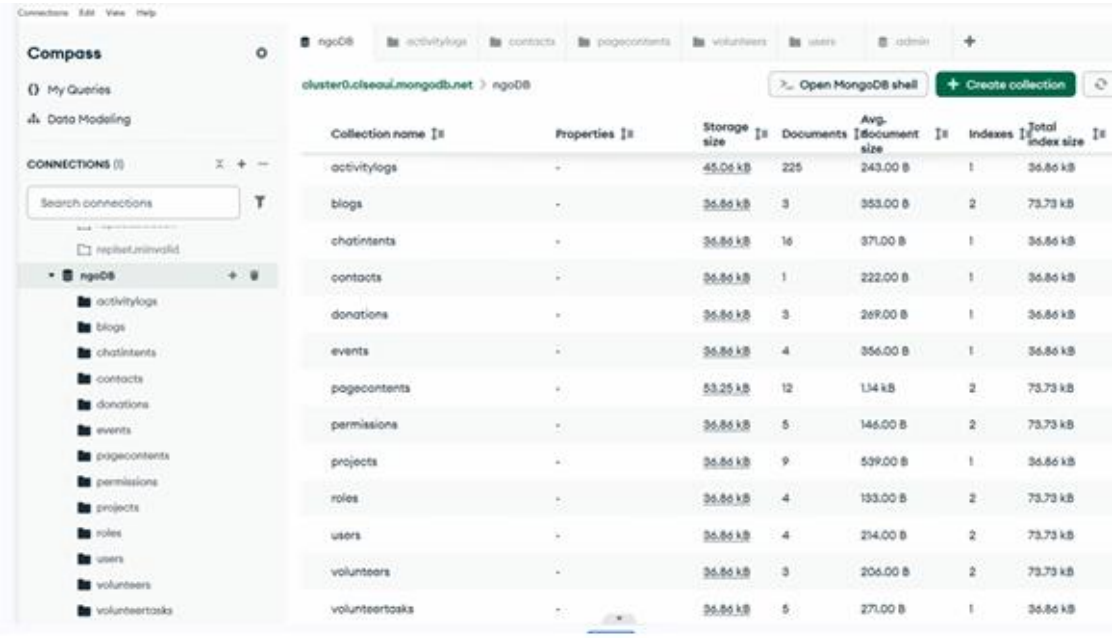
- **ActivityLog.js**: Records an immutable audit trail of administrative modifications. If an Admin deletes a project, this schema captures the Admin ID, the action type, the timestamp, and the target, ensuring total accountability.
- **User.js**: The system-wide identity schema. Holds critical secure data including hashed passwords and explicitly unique emails. Crucially, it links via an ObjectId ref to a specific Role document, dictating their entire access scope.

- **Role.js & Permission.js**: Macro-identifier schemas defining functional buckets of power (e.g., "Admin", "User", "Manager") and extremely granular privileges bound to them.
- **Project.js & Donation.js**: Schemas managing initiatives and tracking financial contributions linked to specific donors.
- **Volunteer.js & VolunteerTask.js**: Tracks metadata specific to applicants (skills, vetting) and atomic units of task operations assigned.
- **Blog.js & Event.js**: Schemas for managing marketing objects like articles and calendar schedules.
- **Contact.js & Document.js & ChatIntent.js & PageContent.js**: Generic data stores for CMS content, uploaded metadata, and specific application modules.

Backend connectivity

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI, {
      tls: true,
      tlsAllowInvalidCertificates: true
    });
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Backend outcome



Controllers (backend/controllers)

The business logic core of the application. Controllers act as the orchestrators: processing incoming HTTP network requests from routes, applying complex business

rules spanning multiple Models, handling unexpected exceptions, and formatting the final JSON response sent back to the client.

- **authController.js**: Highly specialized and sensitive. Responsible exclusively for authentication flows: registering new entries, securely hashing passwords with bcrypt, checking incoming plaintext passwords against hashed records during login, and finally, minting cryptographically signed, time-limited JSON Web Tokens (JWTs).

- **projectController.js**: Core handler managing all modifications concerning Project database operations, validating that total sums or goals aren't mathematically impossible before saving.
- **volunteerController.js**: Handles complex volunteer workflows ranging from parsing public application form schemas, recording admin vetting decisions safely, and allocating specific task assignments natively.
- **adminController.js & securityController.js & donationController.js & pageContentController.js etc...**

Controller sample codes

```
const Project = require('../models/Project');

// @desc    Get all Projects (with filters)
// @route   GET /api/projects
// @access  Public
const getProjects = async (req, res) => {
    try {
        const { category, status } = req.query;
        let query = {};

        if (category) query.category = category;
        if (status) query.status = status;

        // Custom sort order for Status: Upcoming -> Ongoing -> Completed
        // MongoDB doesn't do custom enum sort easily without aggregation
        // Simple approach: Fetch all and sort in JS if pagination not h
        // OR use a numeric priority field.
        // Let's rely on standard sort for now, or fetch and sort.
        // "Show Upcoming first". Upcoming > Ongoing > Completed.

        let projects = await Project.find(query).sort({ updatedAt: -1 })

        // Manual Sort for Status
        const statusOrder = { 'Upcoming': 1, 'Ongoing': 2, 'Completed':
        projects.sort((a, b) => {
            return (statusOrder[a.status] || 99) - (statusOrder[b.status
        });

        res.json(projects);
    } catch (error) {
        res.status(500).json({ message: error.message });
```

```
const { OAuth2Client } = require('google-auth-library');
const client = new OAuth2Client(process.env.GOOGLE_CLIENT_ID);

const generateToken = (id) => {
    // Use fallback if env var is missing to prevent crash
    const secret = process.env.JWT_SECRET || 'fallback_secret_key_123';
    return jwt.sign({ id }, secret, {
        expiresIn: '30d',
    });
};

// @desc    Google Login
// @route   POST /api/auth/google
// @access  Public
const googleLogin = async (req, res) => {
    try {
        const { token } = req.body;
        const ticket = await client.verifyIdToken({
            idToken: token,
            audience: process.env.GOOGLE_CLIENT_ID,
        });
        const { name, email, sub } = ticket.getPayload();

        let user = await User.findOne({ email });

        if (!user) {
            // Create new user
            user = await User.create({
                name,
                email,
                googleId: sub,
```

```
const PageContent = require('../models/PageContent');

// @desc    Get Page Content by Key or Section
// @route   GET /api/content/:section
// @access  Public
const getContentBySection = async (req, res) => {
    const { section } = req.params;

    try {
        const contents = await PageContent.find({ section });
        // Return object with key-value pairs for easier frontend usage
        const contentMap = {};
        contents.forEach(item => {
            contentMap[item.key] = item.value;
        });
        res.json(contentMap);
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
};

module.exports = {
    getContentBySection
};
```

Routes (backend/routes)

Express.js Router instances. These independent scripts act as the traffic controllers. They explicitly link URL signatures/verbs (e.g., GET /api/projects/:id) to their corresponding controller function algorithms while dictating the strict linear sequence of middleware applied to that specific request path.

- **authRoutes.js**: Exposes the entirely public REST endpoints /api/auth/register and /api/auth/login, applying strict Rate Limiter middleware specifically here to thwart brute forces, pointing logic to the authController.

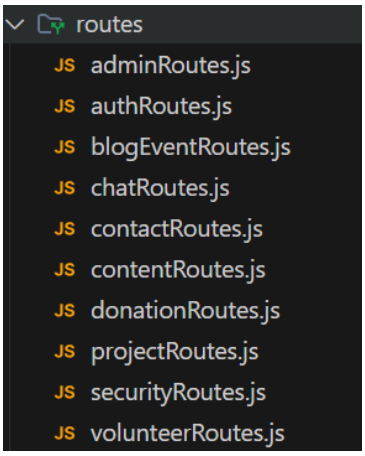
- **adminRoutes.js & projectRoutes.js & securityRoutes.js etc...**

Utilities & Middleware (backend/middleware & backend/utils)

These are powerful helper configurations and request interceptor systems guarding, standardizing, or simplifying the business flows of the application.

- **middleware/authMiddleware.js**: The primary API security chokepoint. Specifically contains the protect function that forcibly evaluates incoming Authorization headers. It mathematically verifies the JWT using the server's hidden secret key, finds the associated user in the Database, attaches them to the req object, and only then relinquishes control to the next chain logic.
- **middleware/roleMiddleware.js**: Houses the subsequent, deeper security handlers checkRole and checkPermission. Once authMiddleware validates sheer *existence*, this file evaluates explicit *authority*.
- **utils/activityLogger.js**: Isolates standard database-write audit logging logic into a simplified, repeatable function allowing specific controllers to inject "Actions" asynchronously into the system-wide ActivityLog.

Routes sample code



```
const express = require('express');
const router = express.Router();
const {
    updatePageContent,
    uploadImage,
    createProject,
    updateProject,
    updateProjectStatus,
    getVolunteers,
    verifyVolunteer,
    assignTask
} = require('../controllers/adminController');
const { protect } = require('../middleware/authMiddleware');
const { checkRole } = require('../middleware/roleMiddleware');
const upload = require('../middleware/uploadMiddleware');

// All routes here are protected and require Admin role
router.use(protect);
router.use(checkRole('Admin'));

// Content
router.put('/content', updatePageContent);
router.post('/upload', upload.single('image'), uploadImage);

// Projects
router.post('/projects', createProject);
router.put('/projects/:id', updateProject); // Full update
router.put('/projects/:id/status', updateProjectStatus);

// Volunteers
router.get('/volunteers', getVolunteers);
router.put('/volunteers/:id/verify', verifyVolunteer);
```

```
const express = require('express');
const router = express.Router();
const { registerUser, loginUser, getMe, googleLogin } = require('../contro
const { protect } = require('../middleware/authMiddleware');

router.post('/register', registerUser);
router.post('/login', loginUser);
router.post('/google', googleLogin);
router.get('/me', protect, getMe);

const { checkRole } = require('../middleware/roleMiddleware');
router.get('/admin-only', protect, checkRole('Admin'), (req, res) => {
    res.send('Admin access granted');
});

module.exports = router;
```

```
const express = require('express');
const router = express.Router();
const securityController = require('../controllers/securityController')
const { protect } = require('../middleware/authMiddleware');
const { checkRole } = require('../middleware/roleMiddleware');

// Require Admin role for all security management endpoints
router.use(protect, checkRole('Admin'));

// Roles
router.get('/roles', securityController.listRoles);
router.get('/roles/:id', securityController.getRole);
router.post('/roles', securityController.createRole);
router.put('/roles/:id', securityController.updateRole);
router.delete('/roles/:id', securityController.deleteRole);

// Permissions
router.get('/permissions', securityController.listPermissions);
router.get('/permissions/:id', securityController.getPermission);
router.post('/permissions', securityController.createPermission);
router.put('/permissions/:id', securityController.updatePermission);
router.delete('/permissions/:id', securityController.deletePermission);

module.exports = router;
```

11. IMPLEMENTATION

The implementation phase involves developing the **NGO Management System** using a modular architecture with clearly separated frontend and backend components. The platform focuses on improving social impact by providing transparent donation management and efficient communication between administrators, volunteers, and donors. It enables administrators to manage projects, process donations, and coordinate volunteer activities efficiently.

Frontend Development

The frontend is developed using **React with Vite** to ensure fast rendering and a responsive user interface. The interface is styled using **Tailwind CSS**, which allows flexible and responsive design across different screen sizes.

UI Design

- Create pages for exploring ongoing projects (**Projects**), reading updates (**Blogs & Events**), and contributing to social causes (**Donate**).
- Implement responsive and accessible layouts that function effectively on both desktop and mobile devices.

Core Features

- **User Authentication**
    - Implement secure login and registration functionality.
    - Provide input validation for user credentials.
    - Integrate **Google OAuth** for simplified user authentication.
- **Donation Processing**
    - Allow donors such as corporate organizations, government bodies, and individual contributors to submit donations.
    - Provide a system for tracking donation status and maintaining transparency.
- **Dashboard Integration**
    - Provide dashboards for administrators and volunteers to manage projects, activities, and system roles.
    - Allow administrators to monitor donations, volunteers, and project updates.

- **Volunteer Management**
    - Allow volunteers to register through the platform.
    - Enable volunteers to view assigned tasks and update their activity logs.

Testing

- Validate user interface components such as donation forms and contact forms.
- Ensure cross-browser compatibility across modern browsers such as **Chrome, Firefox, and Safari**.

Backend Development

The backend is developed using **Node.js and Express.js**, which handle API requests, authentication processes, and communication with the database.

API Development

- **CRUD Operations**
    - Implement functionality to create, read, update, and delete records related to projects, donations, volunteers, blogs, and events.
- **Authentication**
    - Secure user login and registration using **JWT tokens**.
    - Implement password hashing using **bcryptjs**.
- **Role-Based Access Control**
    - Define different access levels for **Admin, Volunteer, and Donor** roles.
    - Restrict system operations based on user permissions.

Database Setup

The system uses **MongoDB with Mongoose** for database management. The following schemas were designed to store application data:

- **Users**
    - { name, email, password (hashed), role, isVerified }
- **Projects**

    - { title, description, category, status (Upcoming, Ongoing, Completed), location }

- **Donations**
    - { category, amount, donor details (name, email, PAN), status }

- **Volunteers**
    - { user details, assigned tasks, activity logs }

Backend Testing

- Validate API endpoints such as /projects and /donations using **Postman**.
- Test database queries for performance, including filtering projects by category and tracking donation status.

Testing API

1. API Testing (Postman)

- Verified all API routes.
- Tested authentication endpoints such as /login and /register.
- Verified project management endpoints (/projects) and donation tracking endpoints (/donations).
- Tested error handling for:
    - Invalid tokens
    - Unauthorized access
    - Missing required fields

2. Unit Tests

- **Backend**
    - Tested password hashing using **bcryptjs**.
    - Validated JWT generation and authentication logic.
- **Frontend**
    - Tested key UI components such as:

- Donation submission forms
- User login forms

3. Integration Tests

- Verified the complete donation workflow
  (User Interface → API → MongoDB → Dashboard Update).
- Tested contact and inquiry submission
  (Form submission → Database logging → Admin dashboard display).

4. User Testing

- Conducted beta testing with volunteers and administrators.
- Verified project creation, donation tracking, and dashboard navigation.

5. Error Handling

- Invalid inputs such as incorrect **PAN formats** or incomplete volunteer registrations trigger clear descriptive error messages.

Results

The implementation of the NGO Management System was successfully completed and tested. All major modules including project management, donation processing, volunteer management, and authentication worked as expected. Minor interface improvements were made after feedback from users during testing. The system provides a reliable platform for NGOs to manage activities, engage volunteers, and maintain transparency in donations.

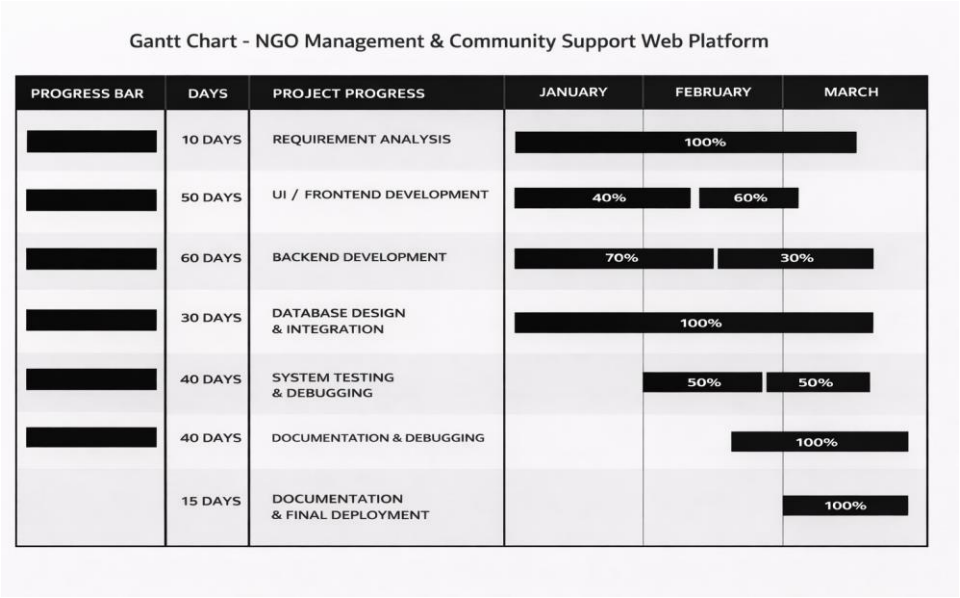
12. PLANNING SCHEDULE

A **Gantt Chart** was used to illustrate project tasks, timelines, and responsibility allocation. This helped ensure effective planning and smooth progress tracking throughout the development of the **NGO Management & Community Support Web Platform**.

Purpose of the Gantt Chart in this Project

- **Task Scheduling**
  - Defines the start and end dates for each development phase including system design, frontend development, backend implementation, and database configuration.
- **Progress Monitoring**
  - Allows tracking of task completion status to ensure that the project progresses according to the planned schedule and meets development deadlines.
- **Task Allocation**
  - Helps assign responsibilities among team members including system designers, developers, and testers to ensure efficient collaboration.
- **Milestone Tracking**
  - Highlights important milestones such as:
    - User authentication implementation
    - Volunteer and donation management modules
    - Beneficiary request handling
    - System deployment

The Gantt Chart was created using **Canva**, which helped visualize the project timeline clearly. It assisted in identifying potential delays, optimizing task distribution, and improving overall project transparency. This approach ensured a well-organized and efficient development process for the **NGO Management & Community Support Web Platform**.


Gantt Chart - NGO Management & Community Support Web Platform
PROGRESS BAR
DAYS
PROJECT PROGRESS
JANUARY
FEBRUARY
MARCH
10 DAYS
REQUIREMENT ANALYSIS
100%
50 DAYS
UI / FRONTEND DEVELOPMENT
40%
60%
60 DAYS
BACKEND DEVELOPMENT
70%
30%
30 DAYS
DATABASE DESIGN & INTEGRATION
100%
40 DAYS
SYSTEM TESTING & DEBUGGING
50%
50%
40 DAYS
DOCUMENTATION & DEBUGGING
100%
15 DAYS
DOCUMENTATION & FINAL DEPLOYMENT
100%

13. TESTING

13.1 Test Cases

Test Case ID	Test Description	Steps to Execute	Test Data	Expected Result	Actual Result	Status
TC-001	Verify Homepage Loading	1. Open the browser. 2. Enter the website URL. 3. Press Enter.	URL: https://ngo-indol.vercel.app	Homepage should load successfully with NGO and Volunteer signup options visible.	Homepage loads successfully with all required sections visible.	Pass
TC-002	Verify NGO Signup Button	1. Navigate to the homepage. 2. Locate the "Sign Up your NGO Now" button. 3. Click the button.	N/A	System should redirect the user to the NGO registration page.	User is redirected to the NGO signup form page successfully.	Pass
TC-003	Verify Volunteer Signup Button	1. Navigate to the homepage. 2. Locate the "Sign Up as Volunteer" button.	N/A	Volunteer registration form should open.	Volunteer signup form opens successfully.	Pass

		3. Click the button.				
TC-004	Verify Navigation Menu	1. Open the homepage. 2. Click different navigation links in the menu.	N/A	Navigation links should redirect to their respective pages.	All navigation links work correctly.	Pass
TC-005	Verify NGO Registration Form	1. Open NGO signup page. 2. Enter NGO details. 3. Click Submit.	NGO Name, Email	NGO account should be successfully registered.	Form accepts valid data and registers NGO successfully.	Pass
TC-006	Verify Volunteer Registration	1. Open volunteer signup form. 2. Enter volunteer details. 3. Submit the form.	Name, Email, Phone	Volunteer registration should be completed successfully.	Volunteer registration completed successfully.	Pass
TC-007	Validate Empty Fields	1. Open registration form. 2. Leave required fields blank. 3. Click Submit.	Empty Input	System should display validation error messages.	Validation messages displayed correctly.	Pass

TC-008	Validate Email Format	1. Open form. 2. Enter invalid email. 3. Submit form.	abc@invalid	System should reject incorrect email format.	Error message shown for invalid email format.	Pass
TC-009	Validate Phone Number	1. Open form. 2. Enter alphabets in phone number field.	abc123	System should allow only numeric values.	Validation prevents invalid phone input.	Pass
TC-010	Verify Password Masking	1. Open password field. 2. Enter password.	Password123	Password should appear masked (hidden characters).	Password appears masked correctly.	Pass
TC-011	Verify Page Responsiveness	1. Open website on mobile device.	Mobile Resolution	Layout should adjust automatically to different screen sizes.	Responsive design works correctly.	Pass
TC-012	Verify Page Scrolling	1. Open homepage. 2. Scroll the webpage.	N/A	Page should scroll smoothly without layout issues.	Page scrolling works properly.	Pass

TC-013	Verify Footer Section	1. Scroll to bottom of the homepage.	N/A	Footer section should display information and links.	Footer section displayed correctly.	Pass
TC-014	Verify NGO Listings	1. Navigate to NGO listing section.	N/A	NGOs should be displayed in the listing area.	NGO listings displayed correctly.	Pass
TC-015	Verify Volunteer Opportunities	1. Navigate to volunteer opportunities section.	N/A	Volunteer opportunities should be visible.	Opportunities displayed successfully.	Pass
TC-016	Verify Button Click Interaction	1. Click different buttons on the website.	N/A	Buttons should respond and perform expected actions.	Buttons function properly.	Pass
TC-017	Verify Page Refresh	1. Press browser refresh button.	N/A	Page should reload without errors.	Page reloads successfully.	Pass
TC-018	Check Broken Links	1. Navigate through all pages. 2. Click all links.	N/A	No links should return errors or broken pages.	All links working correctly.	Pass

TC-019	Verify Form Data Retention	1. Enter data in form fields. 2. Do not submit form.	Sample User Data	Entered data should remain in form fields.	Data retained successfully.	Pass
TC-020	Verify Browser Compatibility	1. Open website in Chrome. 2. Open website in Edge.	Browsers	Website should function properly in different browsers.	Works correctly in tested browsers.	Pass
TC-021	Verify Loading Speed	1. Open homepage.	N/A	Page should load within a few seconds.	Fast loading observed.	Pass
TC-022	Verify UI Consistency	1. Navigate through multiple pages.	N/A	UI layout and design should remain consistent.	UI consistency maintained across pages.	Pass
TC-023	Verify Error Handling	1. Enter incorrect form data. 2. Submit form.	Invalid Data	System should display appropriate error message.	Error message displayed correctly.	Pass
TC-024	Verify CTA Button Visibility	1. Open homepage. 2. Observe signup buttons.	N/A	CTA buttons should be clearly	Buttons clearly visible.	Pass

				visible to users.		
TC-025	Verify Overall System Functionality	1. Navigate through entire website.	N/A	All main features should function properly.	System functions correctly without errors.	Pass

14. FUTURE ENHANCEMENTS & CONCLUSIONS

The aim of these enhancements is to improve user experience, streamline daily operations, and expand the platform's capabilities. Each feature is designed to address specific user needs and optimize the overall functionality of the NGO Management System:

- **Advanced Search and Filtering for Projects and Events:** Enhancing search and filtering capabilities will allow users and potential volunteers to refine project and event options based on criteria such as location, cause category, and date. This will help users find suitable volunteer opportunities or projects to donate to quickly and efficiently.
- **Mobile App Development (iOS & Android):** Expanding the platform to native mobile applications will improve accessibility and engagement. A dedicated mobile app will allow volunteers to manage their tasks easily on the go, enable donors to make quick contributions, and allow all users to receive real-time push notifications seamlessly.
- **AI-Driven Project Recommendations:** Implementing AI-powered recommendations will help users discover new NGO projects and volunteer opportunities that best match their interests based on their browsing history, previous donations, and volunteer profiles.
- **Automated Follow-Up System:** Integrating an automated follow-up system using email or messaging services will help remind volunteers of upcoming event tasks, acknowledge pending volunteer applications, and send regular updates to recent donors, ensuring better community engagement.
- **Multilingual Support & Accessibility Features:** Introducing multilingual support will make the platform more inclusive, catering to a wider and more diverse audience. Accessibility improvements such as screen reader compatibility, keyboard navigation, and high-contrast modes will ensure usability for individuals with disabilities.

14.1 Conclusion

The NGO Management System was developed to enhance community engagement and streamline administrative operations for the organization. By offering a seamless donation portal, an organized volunteer management system, and responsive chatbot assistance, the platform simplifies the entire process of getting involved and improves accessibility for donors, volunteers, and administrative staff alike.

With features such as role-based dashboards, secure authentication, event management, and activity tracking, the platform enables users to efficiently manage their engagements and contributions. The AI chatbot and organized data structures further enhance user support, ensuring a smooth and informative experience.

Future enhancements, including native mobile app development, advanced filtering features, and automated follow-up systems, will further refine the platform and expand its reach. As the system evolves, it has the potential to become a vital tool in the non-profit sector, making charitable workflows more accessible, user-friendly, and efficient for the community, volunteers, and administrators.